

```
In [5]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
df = sns.load_dataset('titanic')
print(df.head())
print(df.isnull())
print(df.info())
print(df.describe())
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class
0	0	3	male	22.0	1	0	7.2500	S	Third
1	1	1	female	38.0	1	0	71.2833	C	First
2	1	3	female	26.0	0	0	7.9250	S	Third
3	1	1	female	35.0	1	0	53.1000	S	First
4	0	3	male	35.0	0	0	8.0500	S	Third

	who	adult_male	deck	embark_town	alive	alone
0	man	True	NaN	Southampton	no	False
1	woman	False	C	Cherbourg	yes	False
2	woman	False	NaN	Southampton	yes	True
3	woman	False	C	Southampton	yes	False
4	man	True	NaN	Southampton	no	True

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class
0	False	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False	False
..	...	...	...	...	...	...	...	...	...
886	False	False	False	False	False	False	False	False	False
887	False	False	False	False	False	False	False	False	False
888	False	False	False	True	False	False	False	False	False
889	False	False	False	False	False	False	False	False	False
890	False	False	False	False	False	False	False	False	False

	who	adult_male	deck	embark_town	alive	alone
0	False	False	True	False	False	False
1	False	False	False	False	False	False
2	False	False	True	False	False	False
3	False	False	False	False	False	False
4	False	False	True	False	False	False
..	...	...	...	...	...	...
886	False	False	True	False	False	False
887	False	False	False	False	False	False
888	False	False	True	False	False	False
889	False	False	False	False	False	False
890	False	False	True	False	False	False

[891 rows x 15 columns]

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 891 entries, 0 to 890

Data columns (total 15 columns):

#	Column	Non-Null Count	Dtype
0	survived	891 non-null	int64
1	pclass	891 non-null	int64
2	sex	891 non-null	object
3	age	714 non-null	float64
4	sibsp	891 non-null	int64
5	parch	891 non-null	int64
6	fare	891 non-null	float64
7	embarked	889 non-null	object
8	class	891 non-null	category
9	who	891 non-null	object
10	adult_male	891 non-null	bool
11	deck	203 non-null	category
12	embark_town	889 non-null	object

```

13 alive      891 non-null    object
14 alone      891 non-null    bool
dtypes: bool(2), category(2), float64(2), int64(4), object(5)
memory usage: 80.7+ KB
None

```

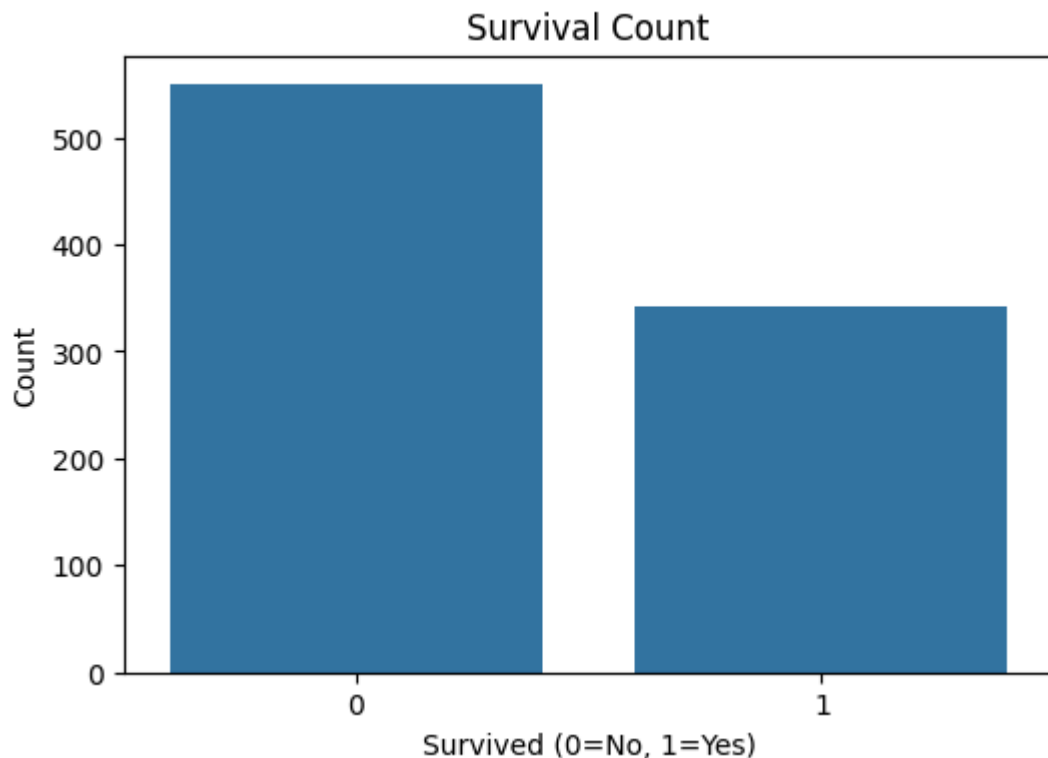
	survived	pclass	age	sibsp	parch	f
count	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

1. Survival Bar Chart

```

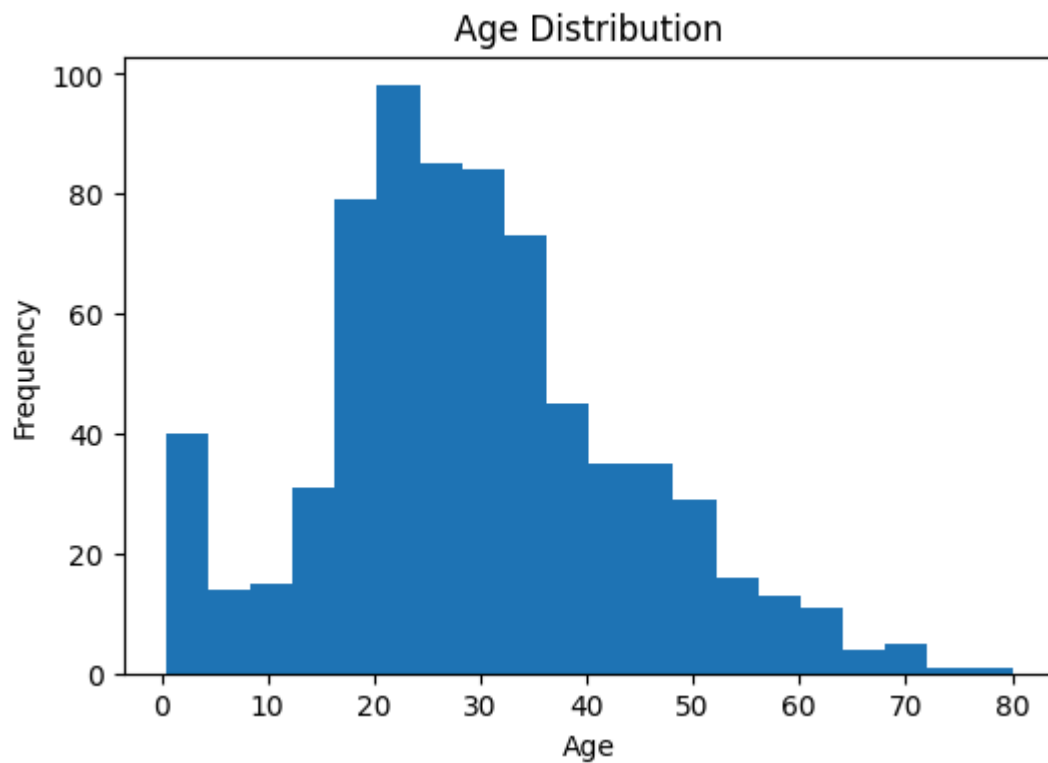
In [6]: plt.figure(figsize=(6,4))
sns.countplot(data=df, x='survived')
plt.title('Survival Count')
plt.xlabel('Survived (0=No, 1=Yes)')
plt.ylabel('Count')
plt.show()

```



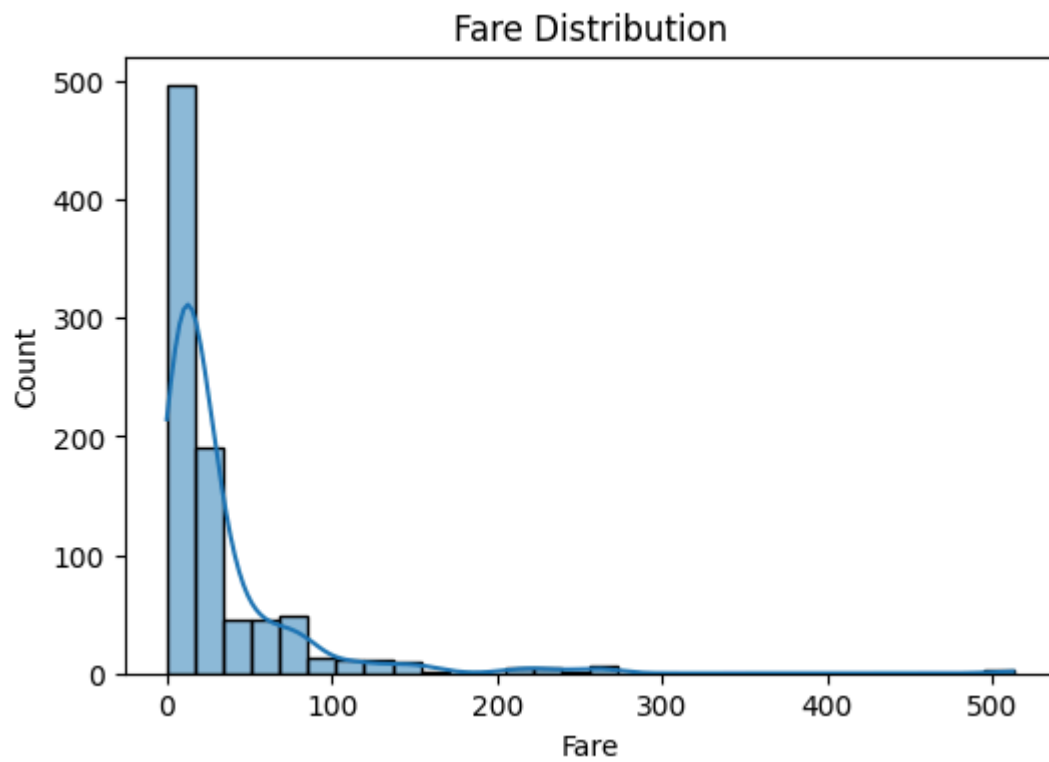
2. Age Histogram

```
In [7]: plt.figure(figsize=(6,4))
plt.hist(df['age'].dropna(), bins=20)
plt.title('Age Distribution')
plt.xlabel('Age')
plt.ylabel('Frequency')
plt.show()
```



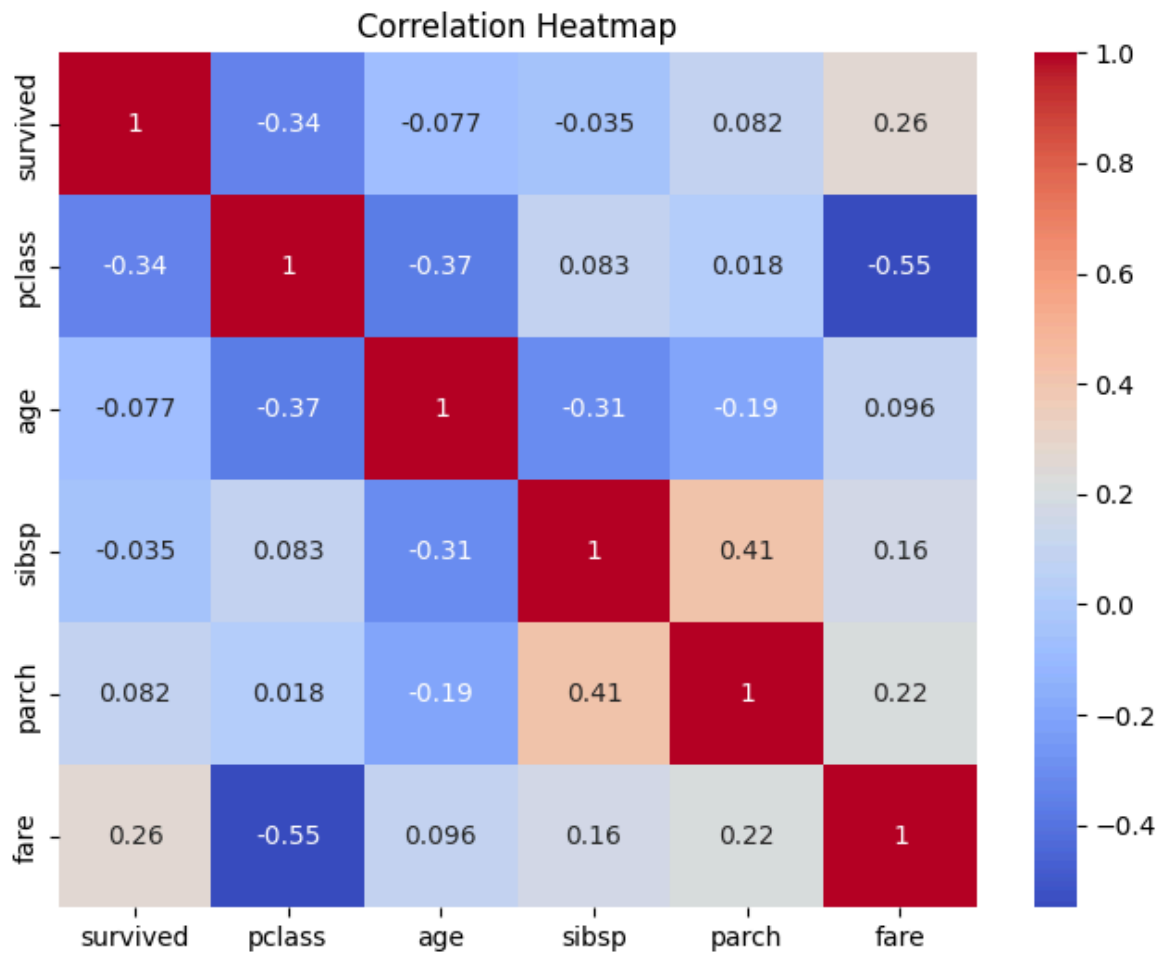
3. Fare Distribution

```
In [8]: plt.figure(figsize=(6,4))
sns.histplot(df['fare'], bins=30, kde=True)
plt.title('Fare Distribution')
plt.xlabel('Fare')
plt.ylabel('Count')
plt.show()
```



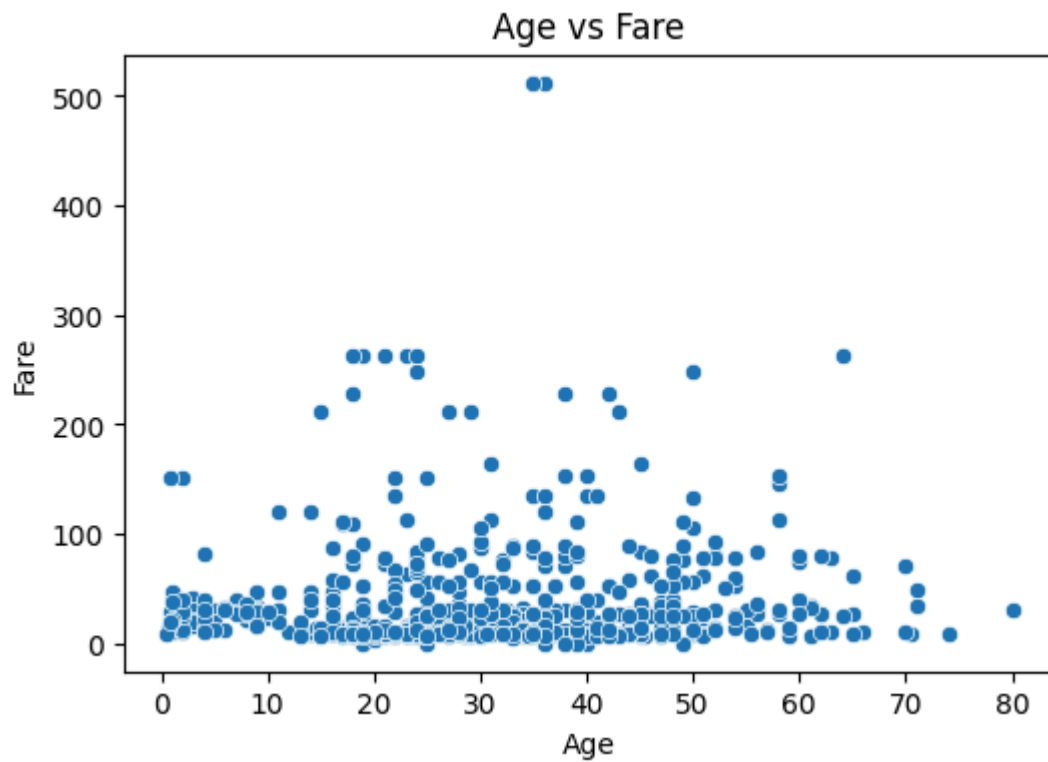
4. Correlation Heatmap

```
In [9]: plt.figure(figsize=(8,6))
numeric_df = df.select_dtypes(include='number')
sns.heatmap(numeric_df.corr(), annot=True, cmap='coolwarm')
plt.title('Correlation Heatmap')
plt.show()
```



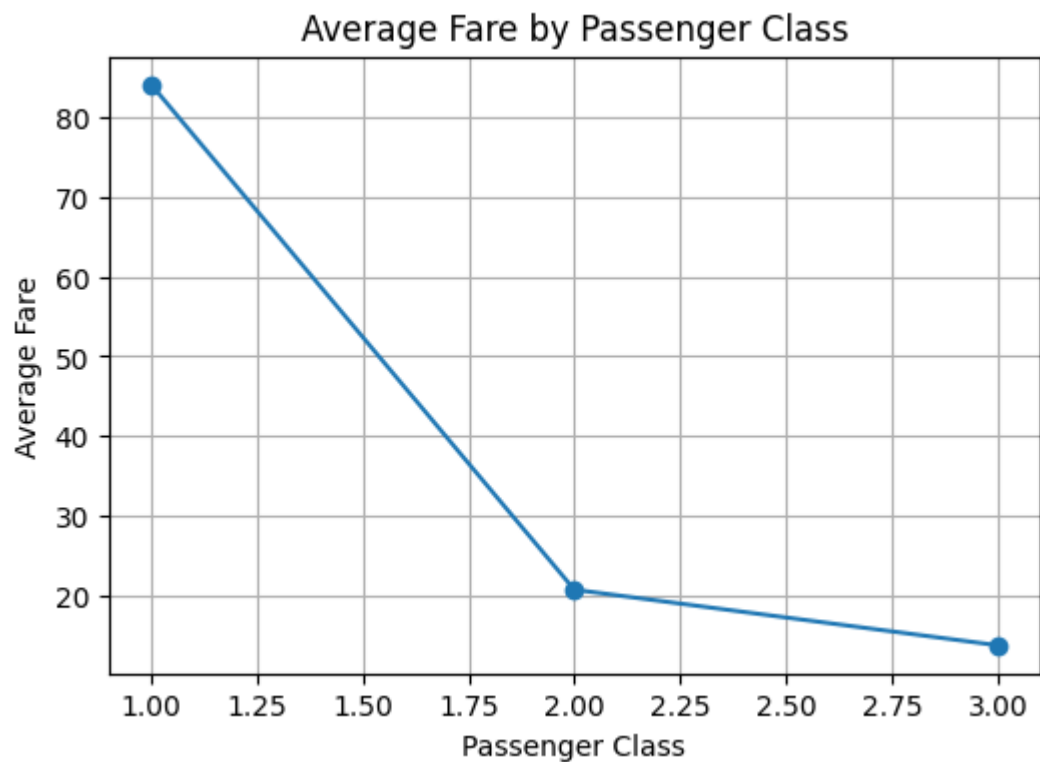
5. Scatter Plot

```
In [10]: plt.figure(figsize=(6,4))
sns.scatterplot(data=df, x='age', y='fare')
plt.title('Age vs Fare')
plt.xlabel('Age')
plt.ylabel('Fare')
plt.show()
```



6. Line Chart

```
In [11]: avg_fare = df.groupby('pclass')['fare'].mean()
plt.figure(figsize=(6,4))
plt.plot(avg_fare.index, avg_fare.values, marker='o')
plt.title('Average Fare by Passenger Class')
plt.xlabel('Passenger Class')
plt.ylabel('Average Fare')
plt.grid(True)
plt.show()
```



central of tendency

mean

```
In [12]: mark=[10,20,30,10]
mean=sum(mark)/len(mark)
print("mean",mean)
d=pd.DataFrame(mark)
print(d.mean())
print(df['age'].mean())
```

```
mean 17.5
0     17.5
dtype: float64
29.69911764705882
```

Median

```
In [13]: import statistics as st
print('median:',st.median(mark))
```

```
median: 15.0
```

Mode

```
In [14]: print('mode:',st.mode(mark))
```

```
mode: 10
```

```
In [15]: print("mean:",d.mean())
print("median:",d.median())
print("mode:",d.mode())
```

```
mean: 0     17.5
dtype: float64
median: 0     15.0
dtype: float64
mode:      0
0     10
```

central of dispresion

```
In [16]: import numpy as np
print("Variance:", d.var(axis=0))
```

```
Variance: 0     91.666667
dtype: float64
```

```
In [17]: print("std:", d.std(axis=0))
```

```
std: 0     9.574271
dtype: float64
```

```
In [18]: data=[5,10,29,34,5,6,7,10]
q1=np.percentile(data,25)
q3=np.percentile(data,75)
iqr=q3-q1
print("iqr:",iqr)
```

```
iqr: 9.0
```


IQR FOR OUTLIER DETECTION $\text{lower} = q1 - 1.5 \cdot \text{iqr}$ $\text{upper} = q3 + 1.5 \cdot \text{iqr}$

```
In [19]: lower=q1-1.5*iqr
         upper=q3+1.5*iqr
         print(lower)
         print(upper)
         out=[x for x in data if x < lower or x > upper]
         print("outliers:",out)
```

-7.75

28.25

outliers: [29, 34]

probability 0=not possible ; 1=possible

```
In [20]: s=(1,2,3,4,5,6)
         e_a=(2,4,6)
         print("sample:",s)
         print("Event A:",e_a)
         p_a=len(e_a)/len(s)
         print("probabilty:",p_a)
```

sample: (1, 2, 3, 4, 5, 6)

Event A: (2, 4, 6)

probabilty: 0.5

In []:

```
In [21]: e_a = set(e_a)

         e_b = {4, 5, 6}
         intersection = e_a.intersection(e_b)

         P_A_given_B = len(intersection) / len(e_b)

         print("Event B:", e_b)
         print("A Intersection B:", intersection)
         print("P(A|B) =", P_A_given_B)
```

Event B: {4, 5, 6}

A Intersection B: {4, 6}

P(A|B) = 0.6666666666666666

```
In [22]: #Example:
         #Disease prevalence=1%
         # Test Sensitivity = 99%
         # Positive test probability= 5%

         P_Disease=0.01
         P_Positive_given_Disease=0.99
         P_Positive=0.05

         P_Disease_given_Positive=(
             P_Positive_given_Disease*P_Disease
         )/P_Positive

         print("\nBayes Theorem Example")
         print("P(Disease)=",P_Disease)
         print("P(Positive|Disease)=",P_Positive_given_Disease)
         print("P(Positive)=",P_Positive)
```

```
print("P(Disease|Positive)=",round(P_Disease_given_Positive,4))
print("Percentage=",round(P_Disease_given_Positive*100,2),"%")
```

Bayes Theorem Example

$P(\text{Disease}) = 0.01$

$P(\text{Positive}|\text{Disease}) = 0.99$

$P(\text{Positive}) = 0.05$

$P(\text{Disease}|\text{Positive}) = 0.198$

Percentage= 19.8 %

```
In [23]: import pandas as pd
df=pd.DataFrame({"hours": [1,2,3,4,5],
                 'mark': [10,20,30,40,50]})
corr=df['hours'].corr(df['mark'])
print("person correlation",corr)
```

person correlation 1.0

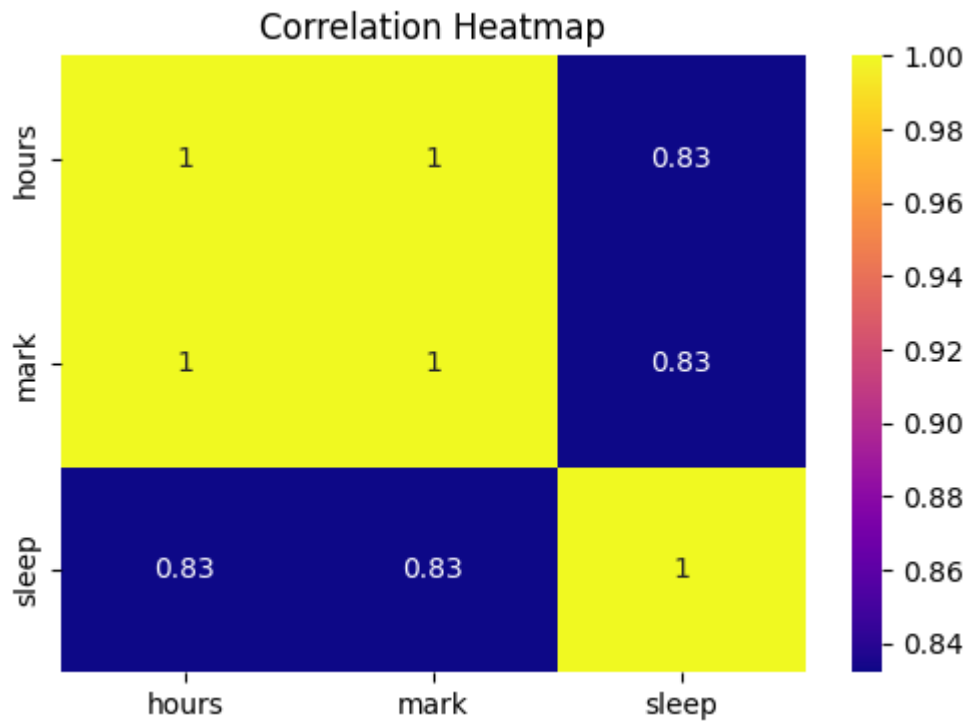
```
In [24]: df=pd.DataFrame({'rank_study':[1,2,3,4,5], 'rank_mark':[2,1,3,5,4]})
corr=df['rank_study'].corr(df['rank_mark'],method='spearman')
print("person correlation",corr)
```

person correlation 0.7999999999999999

```
In [25]: df=pd.DataFrame({"hours": [1,2,3,4,5],
                 'mark': [10,20,30,40,50],
                 'sleep': [6,7,8,9,8]})
print(df.corr)
```

```
<bound method DataFrame.corr of      hours  mark  sleep
0         1    10      6
1         2    20      7
2         3    30      8
3         4    40      9
4         5    50     8>
```

```
In [26]: import matplotlib.pyplot as plt
corr_matrix = df.corr()
plt.figure(figsize=(6, 4))
sns.heatmap(
    corr_matrix,
    annot=True,
    cmap="plasma")
plt.title("Correlation Heatmap")
plt.show()
```

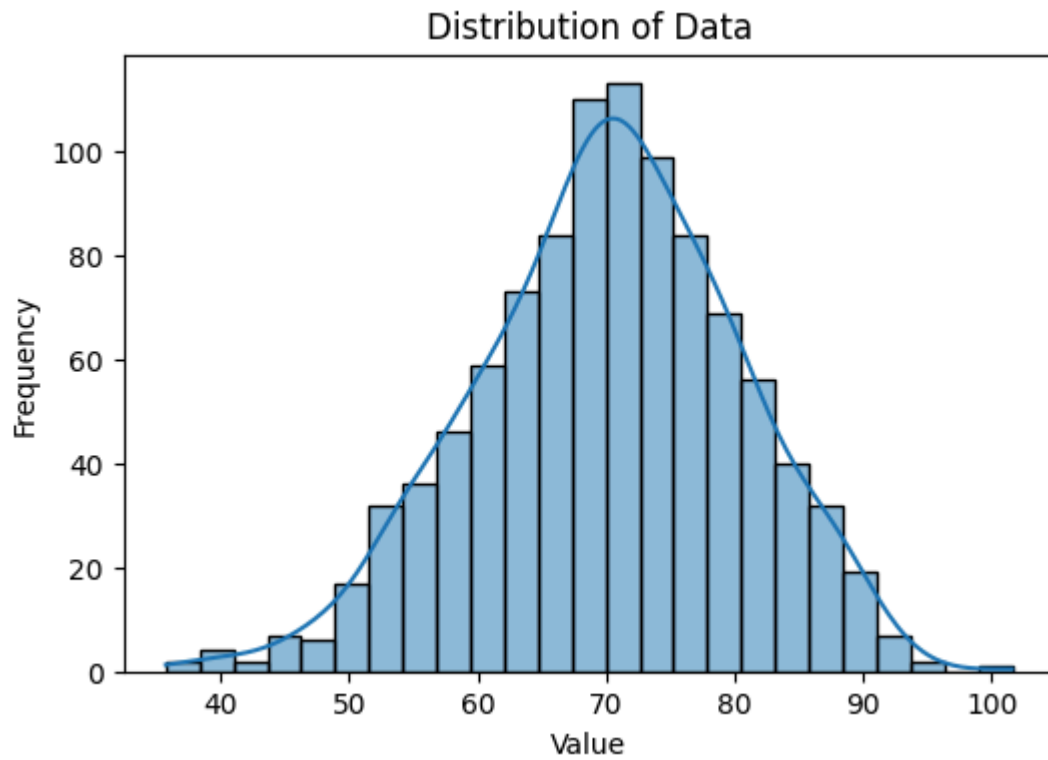


causation-one variable directly cause a change in another variable

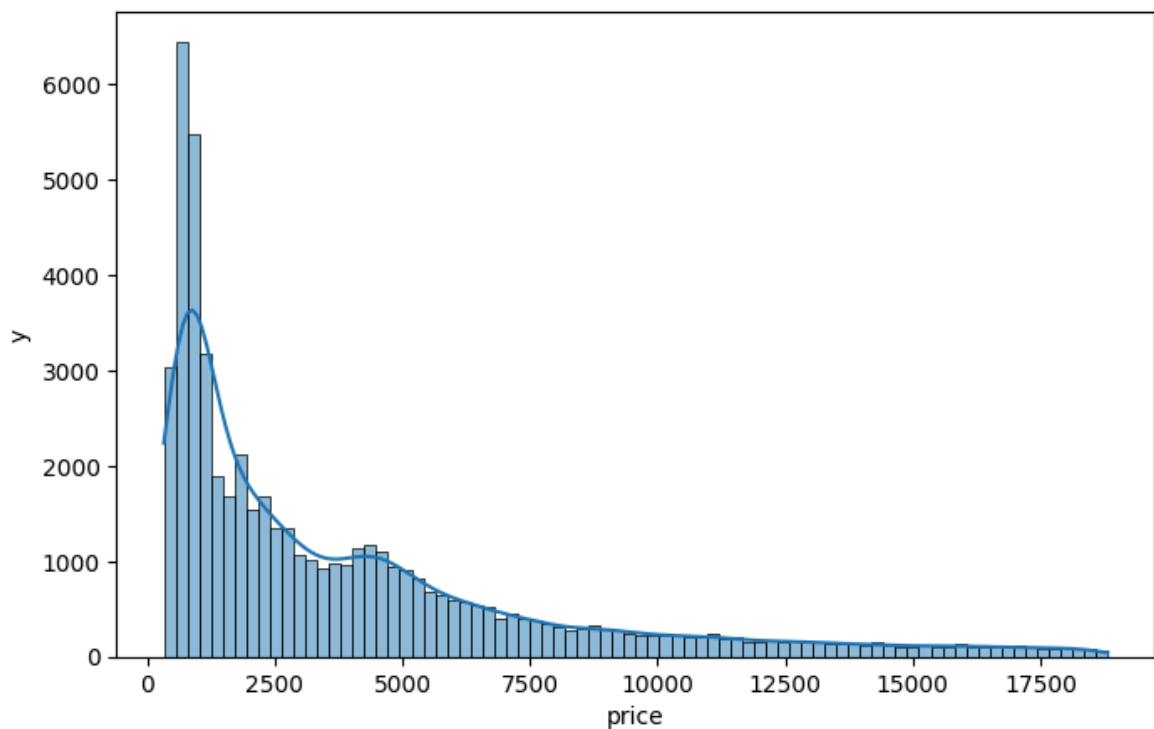
```
In [27]: data=np.random.normal(loc=70,scale=10,size=1000)
print(round(np.mean(data)))
print(round(np.median(data)))
```

70
70

```
In [28]: plt.figure(figsize=(6,4))
sns.histplot(data, kde=True)
plt.title('Distribution of Data')
plt.xlabel('Value')
plt.ylabel('Frequency')
plt.show()
```



```
In [29]: import seaborn as sns
df=sns.load_dataset('diamonds')
plt.figure(figsize=(8,5))
sns.histplot(df['price'],kde=True)
plt.xlabel('price')
plt.ylabel('y')
plt.show()
```



```
In [30]: mean = np.mean(data)
std = np.std(data, ddof=1)
z_scores = (data - mean) / std
z_df = pd.DataFrame({
    'Data': data,
```

```

    'Z_Score': z_scores
})
print(np.std(z_scores, ddof=1))

```

1.0

compute descriptive statistics for a real dataset; identify skewed distribution; apply z-score using it problem; compute pearson and spearman correlation e

```

In [31]: import pandas as pd
import numpy as np
from scipy.stats import zscore, pearsonr, spearmanr
df = pd.read_csv(r"C:/Users/mahim/Downloads/eda and ml dataset/Mental Health Mapping.csv")
mapping = {
    'Days_Indoors': {
        'Go out Every day': 0,
        '1-14 days': 1,
        '15-30 days': 2,
        '31-60 days': 3,
        'More than 2 months': 4
    },
    'Mood_Swings': {'Low': 1, 'Medium': 2, 'High': 3},
    'Growing_Stress': {'No': 0, 'Maybe': 1, 'Yes': 2},
    'Changes_Habits': {'No': 0, 'Maybe': 1, 'Yes': 2},
    'Mental_Health_History': {'No': 0, 'Maybe': 1, 'Yes': 2},
    'Work_Interest': {'No': 0, 'Maybe': 1, 'Yes': 2},
    'Social_Weakness': {'No': 0, 'Maybe': 1, 'Yes': 2}
}
print(mapping)

```

```

{'Days_Indoors': {'Go out Every day': 0, '1-14 days': 1, '15-30 days': 2, '31-60 days': 3, 'More than 2 months': 4}, 'Mood_Swings': {'Low': 1, 'Medium': 2, 'High': 3}, 'Growing_Stress': {'No': 0, 'Maybe': 1, 'Yes': 2}, 'Changes_Habits': {'No': 0, 'Maybe': 1, 'Yes': 2}, 'Mental_Health_History': {'No': 0, 'Maybe': 1, 'Yes': 2}, 'Work_Interest': {'No': 0, 'Maybe': 1, 'Yes': 2}, 'Social_Weakness': {'No': 0, 'Maybe': 1, 'Yes': 2}}

```

```

In [32]: num_df = pd.DataFrame()
for col, mp in mapping.items():
    num_df[col] = df[col].map(mp)

```

```

In [34]: print("DESCRIPTIVE STATISTICS:", num_df.describe())
print("\nSKEWNESS:", num_df.skew())

```

DESCRIPTIVE STATISTICS:		Days_Indoors	Mood_Swings	Growing_Stres
s	Changes_Habits \			
count	292364.000000	292364.000000	292364.000000	292364.000000
mean	1.973516	1.971378	1.023693	1.074729
std	1.409914	0.808396	0.810835	0.817871
min	0.000000	1.000000	0.000000	0.000000
25%	1.000000	1.000000	0.000000	0.000000
50%	2.000000	2.000000	1.000000	1.000000
75%	3.000000	3.000000	2.000000	2.000000
max	4.000000	3.000000	2.000000	2.000000

	Mental_Health_History	Work_Interest	Social_Weakness
count	292364.000000	292364.000000	292364.000000
mean	0.962205	0.929858	0.980309
std	0.819966	0.805599	0.803722
min	0.000000	0.000000	0.000000
25%	0.000000	0.000000	0.000000
50%	1.000000	1.000000	1.000000
75%	2.000000	2.000000	2.000000
max	2.000000	2.000000	2.000000

SKEWNESS: Days_Indoors 0.028970
 Mood_Swings 0.052083
 Growing_Stress -0.043242
 Changes_Habits -0.138278
 Mental_Health_History 0.069823
 Work_Interest 0.127705
 Social_Weakness 0.035587
 dtype: float64

Z-score

```

In [35]: num_df['Days_Indoors_Zscore'] = zscore(num_df['Days_Indoors'])
print("\nZ-SCORE SAMPLE")
print(num_df[['Days_Indoors', 'Days_Indoors_Zscore']].head())
  
```

	Days_Indoors	Days_Indoors_Zscore
0	1	-0.69048
1	1	-0.69048
2	1	-0.69048
3	1	-0.69048
4	1	-0.69048

```

In [36]: outliers = num_df[np.abs(num_df['Days_Indoors_Zscore']) > 3]
print("\nNumber of Outliers:", len(outliers))
pearson_corr, pearson_p = pearsonr(
    num_df['Days_Indoors'],
    num_df['Growing_Stress'])
print("\nPEARSON CORRELATION")
print("Correlation =", pearson_corr)
spearman_corr = num_df['Days_Indoors'].corr(
    num_df['Growing_Stress'],
    method='spearman'
)
print("\nSPEARMAN CORRELATION")
print("Correlation =", spearman_corr)
  
```

Number of Outliers: 0

PEARSON CORRELATION

Correlation = 0.036957789201902166

SPEARMAN CORRELATION

Correlation = 0.03715727487861969